

Data types in Python

16 May 2026 13:29

Number: In number data types stores numerical values only. It is further classified into three different types: int, float and complex.

Type/Class	Description	Examples
int	Integer number	-12, -3, 0, 125, 2
float	Real or floating point numbers	-204.5, 4.0, 14.23
complex	Complex numbers	4+5j, 0j, 2-2j

Boolean data type (bool) is a subtype of Integer. It is a unique data type consisting of two constants, True and False. Boolean True value is non-zero, non-null and non-empty. Boolean False is the value zero.

For example:

```
>>> num1 = 10
>>> type(num1)
<class 'int'>
>>> num2 = -1270
>>> type(num2)
<class 'int'>
>>> var1 = True
>>> type(var1)
<class 'bool'>
>>> f11 = -9.419
>>> type(f11)
<class 'float'>
>>> c = 4 + 5j
>>> type(c)
<class 'complex'>
>>> d = 0j
>>> type(d)
<class 'complex'>
>>>
```

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequence data type available in Python are strings, lists and tuples.

- i. **String:** String is a group of characters. These characters may be alphabet, digits or special characters including spaces. String values are enclosed either in single quotation mark. Or in double quotation mark. For example: 'Hello' or "Hello". The quotes are not a part of the string, they are used to mark the beginning and end of the string for the interpreter. For example:

```
>>> str1 = 'Hello Friends'
>>> str2 = "990"
>>> type(str1)
<class 'str'>
>>> type(str2)
<class 'str'>
>>>
```

- ii. **List:** List is a sequence of items separated by commas and the items are enclosed in brackets[].

For example:

```
>>> fruitList = ['Apple', 'Mangoes', "Grapes", "Litches"]
>>> fruitList
['Apple', 'Mangoes', 'Grapes', 'Litches']
>>> type(fruitList)
<class 'list'>
```

iii. **Tuple:** Tuple is a sequence of items separated by commas and items are enclosed in parenthesis (). This is unlike list where the values are enclosed in brackets []. Once created, we cannot change the tuple.

For example:

```
>>> t = (1, 2, 3.5, 4+5j, 'Hello', 3.6, 'a')
>>> t
(1, 2, 3.5, (4+5j), 'Hello', 3.6, 'a')
>>> type(t)
<class 'tuple'>
```

Set

Set is an unordered collection of items separated by commas, and the items are enclosed and curly brackets {}. A set is similar to list except that it cannot have duplicate entries. Once created, elements of the set cannot be changed. Ordering of elements is not preserved.

For example:

```
>>> set1 = {10, 20.5, "New Delhi"}
>>> type(set1)
<class 'set'>
>>> set1
{'New Delhi', 10, 20.5}
>>>
```

None: None is a special data type with single value. It is used to signify the absence of value in a situation. None supports no special operation and it is neither false nor 0(zero).

For example:

```
myvar = None
type(myVar)

<class 'NoneType'>
```

Mapping: Mapping is an unordered data type in Python. Currently there is only one standard mapping data type in Python called dictionary.

Dictionary: Dictionary in Python holds data items in **key:value** pairs. Items in dictionaries are enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using (:) sign. The key:value pair of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any value in the dictionary we have to specify its key in square brackets, that is [].

```

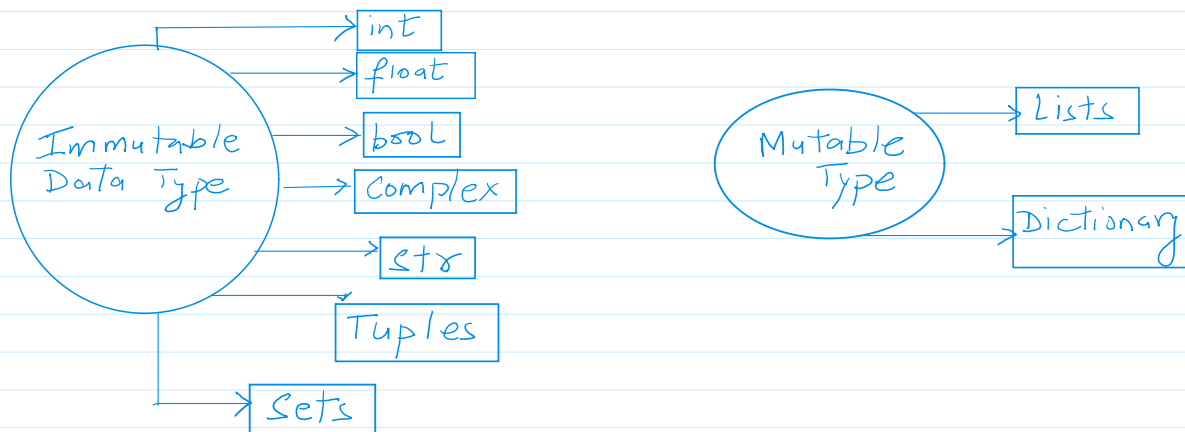
>>> dict1 = {'Fruit': 'Apple', 'Climate': 'cold', 'Price(kg)': 120}
>>> dict1
{'Fruit': 'Apple', 'Climate': 'cold', 'Price(kg)': 120}
>>> dict1['Climate']
'cold'
>>> dict1['Price(kg)']
120

```

Mutable and Immutable Data types

Sometimes we may require to change or update the values of certain variables used in the program. However, for a certain database, Python does not allow us to change the values once a variable of that data type has been created and assigned values.

Variables whose values can be changed after they are created and assigned are called **mutable**. So variables whose values cannot be changed after they are created and assigned are called **immutable**. When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.



Classification of data types

Operators

An operator is used to perform specific mathematical or logical operation on values. The values of the operator work on are called **operands**. For example, in the expression $10 + \text{NUM}$, The value 10 and the variable NUM are operands and the + (plus) sign is an Operator. Python supports several kind of operators whose categorization is briefly explained below:

Arithmetic Operators

Operator	Operation	Description	Example
+	Addition	Adds two numeric values on either side of the operator. This operator can also be used to concatenate two strings on either side of the operator.	<pre> n1 = 5 n2 = 3 r = n1 + n2 r 8 s1 = 'Aarav' s2 = 'Tebrewal' name = s1 + ' ' + s2 name 'Aarav Tebrewal' </pre>

-	Subtraction	Subtract the operand on the right from the operand on the left.	n1 5 n2 3 r = n1 - n2 r 2
*	Multiplication	Multiplies the two values on both side of the operator. Repeat the item on the left side of the operator if first operand is a string and 2nd operand is an integer value.	n1 5 n2 3 p = n1 * n2 p 15 '*' * n1 '*****' n1 * '*' '*****'
/	Division	Divides the operand on the left. By the operand on the right and returns the quotient.	n1 5 n2 3 q = n1 / n2 q 1.6666666666666667
%	Modulus	Divides the operand on the left of by the operand on the right and returns the remainder.	n1 5 n2 3 r = n1 % n2 r 2 r = n1 % 100 r 5
//	Floor Division	Divides the operand on the left by the operand on the right, and returns a quotient by removing the decimal part. It is sometimes also called integer division.	r 5 n1 5 n2 3 r = n1 // n2 r 1
**	Exponent	Performs exponential (power) calculation on operands. That is, raise the operand on the left to the power of the operand on the right.	n1 5 n2 3 r = n1 ** n2 r 125

$$\begin{array}{r} 100 \overline{) 50} \\ \underline{-0} \\ 75 \\ \underline{-75} \\ 0 \end{array}$$
 remainder